ASTAF-R: An Ablation-Guided Spatio-Temporal Framework for Multi-Horizon Traffic Prediction

Shishir Singh Chauhan[1,+], Puneet Nema[2,+], Gunjan Chhabra[3,+], Ankur Pandey[1,*], and Surbhi Syal[1,+]

[1]Manipal University Jaipur, Department of Computer Science and Engineering, Jaipur, 303007, INDIA
[2]Department of Computer Science and Engineering, Lakshmi Narain College of Technology, Bhopal, MP, INDIA
[3]Department of computer science and engineering, School of science and technology, Swami Rama Himalayan University, Dehradun, Uttarakhand, INDIA
[*]ankur.pandey@jaipur.manipal.edu
[+]these authors contributed equally to this work

ABSTRACT

Reliable traffic forecasting comparisons require architectural decisions to be separated from final test evaluation and all models to share the same data pipeline. ASTAF-R is an ablation-guided deterministic framework that combines causal multi-scale temporal encoding, residual signal decomposition, sparse graph propagation, residual spatial attention, and last-observation residual decoding. Candidate variants were selected using validation MAE only, after which the chosen configuration and eight compact common-code comparison models were evaluated once on held-out test periods from METR-LA and PeMS-BAY. Raw timelines were divided chronologically before stride-one 12-to-12 windows were formed, preventing observations from crossing split boundaries. Across seeds 11, 22, and 33, ASTAF-R obtained 2.995$\pm$0.008 mph MAE on METR-LA and 1.542$\pm$0.001 mph on PeMS-BAY. The strongest compact comparison, STID-style, obtained 3.377$\pm$0.011 and 1.709$\pm$0.003 mph, respectively. Calendar-day block bootstrap intervals remained below zero for ASTAF-R minus each comparison model, while seed-level dispersion and resource measurements are reported explicitly. A post-training graph audit found identical uniform top-40 supports across seeds because the graph-source and graph-target parameters converged to zero; consequently, the experiments support sparse graph propagation but not a claim of recovered or seed-robust adaptive topology. Top-k sparsity describes graph connectivity only: the implementation retains dense $N \times N$ graph and attention tensors and therefore makes no sparse-kernel computational claim. The evaluation is limited to two regularly sampled California freeway-speed datasets and compact local baseline implementations. The results therefore establish reproducible performance under this controlled protocol, not universal or official state-of-the-art superiority.

1 Introduction

Traffic-speed forecasting supports congestion warning, route guidance, incident management, and network control. The task is difficult because road measurements combine short-lived transitions, recurring daily structure, spatial dependence, missing observations, and non-stationary disturbances. Modern systems commonly assemble temporal convolutions, graph propagation, attention, and signal decomposition in one architecture. Each component is plausible, but adding plausible components does not by itself establish that they improve held-out forecasting performance.

ASTAF-R treats architecture construction as an empirical selection problem. The starting components are tested through validation-only ablations, and the resulting configuration is frozen before the test set is evaluated. This distinction matters because repeated test-guided refinement can produce an optimistically biased comparison even when every individual run uses a chronological split. The present protocol therefore separates candidate selection, locked testing, and diagnostic analysis.

The input at each forecast origin contains 12 five-minute speed observations from every sensor, and the model predicts the next 12 observations. METR-LA contains 207 sensors and 34,272 time points; PeMS-BAY contains 325 sensors and 52,105 time points. In both cases, the raw timeline is divided into contiguous 70%, 10%, and 20% segments before any windows are generated. This ordering prevents the input or target observations of a window in one split from being reused by a window in another split.

The comparison is intentionally controlled. Persistence and compact STID-, DCRNN-, STGCN-, Transformer-, Graph WaveNet-, AGCRN-, and ASTAF-style models use the same tensors, masks, optimizer family, stopping rule, and test metrics. These implementations are useful common-code controls, but they are not official repository reproductions or claims about the best published configuration of each named method.

The main contributions are:

- a leakage-free raw-timeline split procedure with exact temporal boundary records for two public traffic datasets;
- validation-only selection followed by locked three-seed testing, with seed-aware and calendar-day-block uncertainty analyses;
- a deterministic ASTAF-R implementation combining temporal, decomposition, sparse graph, residual-attention, and residual-decoder components;
- transparent sensitivity and graph diagnostics, including evidence that the learned graph parameters collapse to a uniform sparse support in the selected runs; and
- complete code, configurations, random seeds, run metadata, prediction archives, and environment information for independent reproduction.

The remainder of the paper is organized as follows. Section 2 reviews statistical, recurrent, convolutional, graph, attention, decomposition, and multi-scale forecasting methods. Section 3 defines the multi-horizon task and metrics. Section 4 describes both datasets, preprocessing, and exploratory dependencies. Section 5 specifies ASTAF-R. Section 6 documents the leakage-free protocol, model configurations, and computation environment. Section 7 presents the validation evidence, locked test results, statistical analyses, graph diagnostics, and efficiency trade-offs. Section 8 states the supported conclusions and limitations.

2 Literature Review

Traffic forecasting has a long research history that spans classical statistical methods, deep sequence models, convolutional architectures, graph neural networks, and Transformer-based designs. This section reviews each of these threads in the context of the design decisions made in ASTAF-R, with particular attention to what is known about the individual contributions of attention mechanisms, signal decomposition, and adaptive graph topology learning.

2.1 Classical and Statistical Forecasting Methods

The earliest computational approaches to traffic forecasting were rooted in classical time-series analysis. Autoregressive integrated moving average (ARIMA) and its seasonal variant SARIMA modeled traffic speed and flow as linear functions of their own past observations and residual errors[1]. These models remain valuable in low-dimensional, single-sensor settings where the statistical assumptions of stationarity and linearity approximately hold and where interpretability of the model coefficients is required. Historical average and exponential smoothing approaches provided simpler alternatives with lower computational cost.

The fundamental limitation of classical methods in modern traffic systems is their inability to scale. A road network with N sensors requires modeling N separate time series, and the correlations among those sensors, which carry important information about how congestion propagates spatially, cannot be represented within a univariate framework. Classical vector autoregression can model joint dynamics across multiple series, but its parameter count grows quadratically with N, making it computationally intractable for the sensor counts typical of large urban networks. Non-linear patterns, including sharp congestion onset and rapid clearance, are also beyond the representational capacity of linear statistical models. These limitations created the conditions for deep learning to enter the problem.

2.2 Deep Recurrent and Convolutional Models

The first generation of deep learning approaches to traffic forecasting replaced statistical models with recurrent neural networks. Long short-term memory (LSTM) networks[2] addressed the vanishing gradient problem that limited standard recurrent networks, enabling the learning of dependencies across longer temporal windows. Gated recurrent units (GRU)[3] offered a computationally lighter alternative with comparable representational capacity. Both architectures learn non-linear sequential dependencies and can be stacked or combined with fully connected layers to produce multi-step forecasts.

Convolutional architectures extended deep learning to the spatial dimension in settings where traffic observations could be placed on regular grids. ST-ResNet modeled citywide crowd flows as a sequence of residual convolutional operations applied to grid-based representations[4]. Deep multi-view spatial-temporal networks combined multiple temporal granularities within a single convolutional architecture[5]. Spatio-temporal meta learning frameworks further improved cross-region generalization by learning shared feature representations[6]. These grid-based approaches worked well for problems with regular spatial structure, such as crowd density estimation in urban areas, but road networks are irregular graphs rather than regular grids, and applying grid convolution to sensor data requires imposing an artificial spatial discretization that discards the actual topology of the road network.

2.3 Graph Neural Networks for Traffic Forecasting

Graph neural networks[7,8] provided a natural solution to the topology mismatch problem by representing sensor networks as graphs in which nodes correspond to sensors and edges encode physical or statistical proximity between them. The central operation in a GNN is message passing: each node aggregates feature information from its neighbors according to the graph structure and updates its own representation. For traffic, this means that a sensor in congested conditions can influence the predicted state of nearby sensors, capturing the spatial propagation of congestion that classical and grid-based methods miss.

DCRNN[9] was among the first models to systematically combine graph convolution with recurrent temporal modeling for traffic forecasting. It represented traffic dynamics as a bidirectional diffusion process on the sensor graph, combining encoder-decoder recurrent architectures with diffusion convolution to produce multi-step speed and flow predictions. DCRNN established the benchmark protocol on METR-LA that subsequent methods, including ASTAF-R, continue to use. STGCN[10] replaced the recurrent components with gated temporal convolution, producing a fully convolutional spatio-temporal architecture that was more computationally efficient and easier to train in parallel. T-GCN and related graph recurrent methods explored different combinations of graph convolution and gated recurrent units[11,12]. STSGCN modeled localized spatial-temporal synchronous dependencies by constructing local spatio-temporal graphs within small temporal windows[13].

A recurring limitation of these early GNN-based models was their dependence on a fixed, pre-defined graph topology, typically derived from physical road distances or sensor connectivity. Fixed topology assumptions can fail when the actual statistical dependencies among sensors differ from their geographic proximity, for example when two sensors on different roads share traffic patterns due to common origin or destination zones. Graph WaveNet[14] addressed this by learning an adaptive adjacency matrix from trainable node embeddings, entirely removing the requirement for a pre-defined graph and allowing the model to discover latent spatial dependencies from data. AGCRN[15] extended adaptive graph learning by combining it with node-specific parameter matrices, allowing each sensor to maintain its own recurrent transformation weights while sharing the global graph structure.

Subsequent work has continued to refine adaptive graph construction. Dynamic spatio-temporal graph convolution[16] introduced time-varying adjacency matrices that adapt to different traffic conditions throughout the day. Adaptive neighborhood selection[17] constrained the learned graph to preserve locality properties by limiting the set of neighbors each node can attend to. Graph attention convolution[18] combined learned edge weights with attention-based neighbor selection. Deeper adaptive STGCN[19] stacked multiple adaptive graph convolution layers to capture multi-hop spatial dependencies. Dynamic hypergraph forecasting[20] extended adaptive graph learning to hyperedges that can represent group-level spatial relationships among three or more sensors simultaneously. Fourier-enhanced heterogeneous graph learning[21] combined frequency-domain temporal representations with heterogeneous graph structures to capture both periodic and structural dependencies. Knowledge fusion approaches[22] and open-data graph methods[23] demonstrated the additional value of incorporating external contextual information such as land use, point-of-interest data, and open government records alongside sensor observations.

ASTAF-R evaluates sparse graph propagation within this line of work. Trainable source and target embeddings produce directed scores followed by top-k sparsification and row normalization. Reference, shuffled, uniform, identity, and graph-disabled controls are evaluated explicitly. As reported in Section 7, the selected checkpoints collapse to a uniform top-40 support; the study therefore does not claim recovery of adaptive road topology.

2.4 Attention Mechanisms in Spatio-Temporal Models

Attention mechanisms[24] entered spatio-temporal traffic forecasting as a way to capture dynamic, context-dependent relationships that fixed graph structures and static convolution weights cannot represent. ASTGCN[25] introduced spatial and temporal attention layers on top of a graph convolutional backbone, learning time-varying attention scores that modulate the influence of each neighbor and each historical step on the current prediction. GMAN[26,27] scaled attention to large networks through a multi-head graph attention mechanism interleaved with gated fusion components, achieving strong performance on large benchmark datasets.

The theoretical appeal of attention in traffic forecasting is clear: congestion propagation is non-stationary, and the sensors most relevant to predicting a given location at a given time vary with traffic conditions. An attention mechanism can, in principle, identify and weight the most relevant spatial and temporal contexts dynamically. However, the practical benefit of attention depends on how it interacts with the other components already present in the architecture. Traffic flow attention[28] demonstrated useful performance improvements in specific settings, and dynamic graph convolutional networks with latent topology estimation[29] showed that attention-like mechanisms can implicitly capture time-varying spatial relationships. Yet none of these studies examined whether mandatory attention fusion is beneficial when expressive graph propagation and temporal convolution are simultaneously active.

Attention is treated as a validation-tested component rather than a required source of improvement. Disabled, residual, and mandatory configurations are compared under the same preprocessing and optimization pipeline. The selected configuration uses residual spatial attention, while the conclusions remain specific to the two datasets and declared training budget.

2.5 Signal Decomposition in Temporal Forecasting

Signal decomposition has a long history in time-series analysis. Classical seasonal-trend decomposition methods separate a time series into additive or multiplicative trend, seasonal, and residual components for individual analysis and modeling. In deep learning, this principle was reintroduced with neural implementations in Autoformer[30], which replaced the standard attention operation with a decomposition-aware auto-correlation mechanism that separates trend and seasonal components using moving average kernels before computing temporal dependencies. FEDformer[31] extended this idea by performing decomposition in the frequency domain, combining Fourier and wavelet transforms with sparse attention to separate structured periodic patterns from noise. Both models demonstrated that explicit decomposition consistently improves long-horizon forecasting accuracy on standard benchmarks and reduces the tendency of deep models to over-fit to noise in short-horizon lookback windows.

The benefit of decomposition in traffic-specific spatio-temporal models is less settled. ASTAF-R compares learned fusion with fixed, equal, trend-heavy, and decomposition-disabled alternatives. Their close validation errors do not identify one fusion rule as uniquely effective, so decomposition is presented as an evaluated representation choice rather than a generally optimal module.

2.6 Multi-Scale Temporal Modeling

Traffic dynamics unfold across multiple time scales. Instantaneous speed fluctuations occur at the sensor sampling interval (five minutes in METR-LA), congestion build-up and dissipation typically spans fifteen to thirty minutes, and daily peak patterns repeat over hours. A forecasting model that operates at a single temporal scale will capture only one of these phenomena well and must use depth or capacity to approximate the others.

Multi-scale temporal representations have been explored through dilated causal convolutions[14], multi-branch temporal convolutional blocks, and hierarchical recurrent architectures in prior traffic forecasting work. Graph WaveNet used stacked dilated convolutions with exponentially increasing dilation rates to reach large effective receptive fields within a shallow architecture. ASTAF-R takes a different approach: rather than using dilation to extend a single convolution branch, it applies three parallel convolutions with kernel sizes $k \in \{1,3,6\}$, directly targeting the three most relevant temporal scales for five-minute traffic data, and fuses the scale-specific outputs using a softmax-constrained learnable weight vector. This design is shallower and more interpretable than stacked dilated architectures and provides an explicit fusion mechanism that can be inspected to understand which scale the model relies on most.

Related multifrequency forecasting work studies a different fusion problem. Xiao and Liu[32] integrate heterogeneous exogenous variables sampled at different frequencies for carbon-price prediction. ASTAF-R uses one traffic-speed feature sampled uniformly every five minutes; its "multi-scale" branches are different receptive fields over the same frequency, not heterogeneous-frequency alignment. This distinction narrows the model's data-fusion claim.

2.7 Model Validation and Ablation Practices

Survey studies on deep learning for traffic forecasting[33]and graph neural networks for traffic prediction[34] have consistently identified reproducibility and rigorous component evaluation as methodological gaps in the field. Models are often presented as full packages with comparisons against published baselines, but the contribution of each of the architectural components is rarely isolated. This makes it hard to understand what innovations actually translate into the reported improvements, and whether the same improvements could be obtained with a simpler design.

STID[35] provided a striking counterexample that a simple spatio-temporal identity baseline using only node embeddings and time-of-day features, and no graph convolution or attention, could match or exceed the performance of complex GNN-based models on a number of benchmarks. This result challenged the assumption that architectural complexity is necessary for competitive performance, and inspired more careful ablation practices.

ASTAF-R uses ablation as a hypothesis-generation tool: candidate mechanisms are isolated before being tested under the complete benchmark protocol. The the evidence shows that reduced-sample ablations are not sufficient for component selection, because the attention ranking changes after full-data training. The resulting contribution is therefore an auditable development workflow and evidence trail, not a claim that every initial choice was validated.

2.8 Positioning of ASTAF-R

Table 1 positions ASTAF-R by architectural mechanism. It records presence rather than superiority. The evaluated ASTAF-R is deterministic and contains no uncertainty head.

Table 1. Comparison of representative spatio-temporal traffic forecasting models across key architectural dimensions. ✓ indicates the feature is present; ○ indicates an optional or partial implementation; × indicates the feature is absent.

Model	Adaptive Graph	Temporal Attention	Signal Decomp.	Multi-Scale Encoding
DCRNN[9]	×	×	×	×
STGCN[10]	×	×	×	×
Graph WaveNet[14]	✓	×	×	✓
AGCRN[15]	✓	×	×	×
ASTGCN[25]	×	✓	×	×
GMAN[26]	×	✓	×	×
Autoformer[30]	×	✓	✓	×
FEDformer[31]	×	✓	✓	×
ASTAF (base)	✓	✓	✓	✓
ASTAF-R (ours)	✓	○	✓	✓

ASTAF-R combines mechanisms that also occur individually in earlier models. Its distinguishing research procedure is the explicit connection between pilot hypotheses, complete-data tests, and reported sensitivity results. This does not imply that the resulting configuration is unique or superior to all graph-based forecasting models.

3 Problem Formulation

In this section, we formally define the multi-horizon traffic forecasting problem addressed by ASTAF-R, introduce the mathematical notations used throughout the paper, and describe the evaluation metrics used in all experiments. The evaluated model produces deterministic point forecasts; probabilistic heads are outside the present scope.

3.1 Network and Observation Model

Consider a road network monitored by N traffic sensors deployed at fixed locations. The network is represented as a weighted directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{W})$, where $\mathcal{V}$ is the set of sensor nodes with $|\mathcal{V}| = N$, $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is the set of directed edges encoding physical or statistical proximity between sensor pairs, and $\mathbf{W} \in \mathbb{R}^{N \times N}$ is the weighted adjacency matrix with $W_{ij} > 0$ if sensor j is considered a neighbor of sensor i and $W_{ij} = 0$ otherwise.

At each discrete time step t, the state of the network is described by an observation matrix:

$$\mathbf{X}_t \in \mathbb{R}^{N \times F}, \tag{1}$$

where F is the number of observed features per sensor at each time step. In both METR-LA and PeMS-BAY, $F = 1$ and the single feature is traffic speed. The observation at node i and time t is denoted $x_{i,t} \in \mathbb{R}^F$.

Given a sequence of T consecutive observations ending at the current time step t, the historical input tensor is:

$$\mathbf{X}_{t-T+1:t} \in \mathbb{R}^{T \times N \times F}, \tag{2}$$

where the leading dimension indexes time steps, the second dimension indexes sensors, and the third dimension indexes features. In all verified experiments reported in this paper, $T = 12$, corresponding to one hour of historical observations at five-minute sampling frequency.

3.2 Multi-Horizon Forecasting Problem

The multi-horizon traffic forecasting task is to learn a mapping f_θ parameterized by θ that predicts the future network state over a horizon of H steps:

$$\hat{\mathbf{Y}}_{t+1:t+H} = f_\theta(\mathbf{X}_{t-T+1:t}, \mathcal{G}), \tag{3}$$

where $\hat{\mathbf{Y}}_{t+1:t+H} \in \mathbb{R}^{H \times N \times F'}$ is the predicted output tensor, F' is the number of predicted features per sensor per step, and $\mathcal{G}$ denotes the graph structure available to the model. In ASTAF-R, the graph $\mathcal{G}$ is not fixed to the pre-defined sensor distance matrix; instead, the model learns an implicit adjacency $\mathbf{A}^*$ from trainable node embeddings and blends it with the identity

topology, as described in Section 5. The graph input to Eq. (3) is therefore treated as a learnable component of θ rather than a fixed hyperparameter.

In this study, $H = 12$, corresponding to one hour of prediction at five-minute intervals. The model therefore ingests 12 steps of observed speed across 207 sensors and simultaneously outputs 12 future steps of predicted speed for all 207 sensors. This simultaneous multi-step formulation differs from recursive single-step prediction schemes in that all H horizon steps are produced in a single forward pass without compounding individual step errors across iterations.

For the batch-level computation used during training and evaluation, the input tensor is augmented with a batch dimension B:

$$\mathbf{X} \in \mathbb{R}^{B\times T\times N\times F}, \qquad \hat{\mathbf{Y}} \in \mathbb{R}^{B\times H\times N\times F'}, \tag{4}$$

where B is the number of samples in each mini-batch. This four-dimensional tensor format is used consistently throughout the ASTAF-R architecture description in Section 5.

3.3 Training Objective

The model parameters θ are optimized by minimizing masked mean absolute error between predicted and observed speed values over all training samples, sensors, and horizon steps:

$$\mathscr{L}(\theta) = \frac{\sum_{i=1}^{M} m_i \, |y_i - \hat{y}_i|}{\sum_{i=1}^{M} m_i}, \tag{5}$$

where M is the total number of scalar prediction targets, $m_i \in \{0,1\}$ excludes unavailable targets, across all samples, sensors, and horizon steps in the training set, y_i is the i-th ground-truth normalized speed observation, and $\hat{y}_i$ is the corresponding model prediction. The MAE objective in Eq. (5) is used because it is robust to outliers in traffic speed data, which arise frequently from incident-affected sensors, it treats over- and under-prediction symmetrically, and it corresponds directly to the primary evaluation metric used throughout this paper. Accordingly, Eq. (5) serves a dual role: it is both the loss function minimized during training and the MAE evaluation metric reported in all experimental results. No separate definition of the MAE evaluation metric is needed; all references to MAE in Sections 6 and 7 refer to the same quantity defined here.

ASTAF-R produces deterministic point forecasts. Probabilistic forecasting is outside the evaluated contribution. A future extension would require explicit distributional or quantile training and evaluation with coverage, interval width, calibration error, and proper scoring rules.

3.4 Evaluation Metrics

Three complementary metrics are used to assess forecasting performance across all experimental configurations. Each captures a different aspect of prediction quality, and together they provide a complete characterization of model behavior across the 12-step prediction horizon.

Mean Absolute Error (MAE). Equation (5) defines MAE as the average magnitude of prediction errors regardless of direction. Because it is linear in error size and treats over- and under-prediction equally, it serves as the primary metric. Metrics are evaluated after inverse transformation into the original speed scale. Thus, an MAE of δ means that the average valid forecast differs from the observation by δ miles per hour.

Root Mean Square Error (RMSE). RMSE measures the square root of the average squared prediction error:

$$\text{RMSE} = \sqrt{\frac{1}{M}\sum_{i=1}^{M}(y_i - \hat{y}_i)^2}. \tag{6}$$

RMSE is more sensitive than MAE to large individual errors because it squares each residual before averaging. Such errors often occur during the onset or clearance of congestion, when speed changes abruptly. RMSE is therefore reported with MAE to show whether a model controls these larger deviations.

Mean Absolute Percentage Error (MAPE). MAPE expresses the average prediction error as a percentage of the observed value:

$$\text{MAPE} = \frac{1}{M}\sum_{i=1}^{M}\left|\frac{y_i - \hat{y}_i}{y_i + \varepsilon}\right|, \tag{7}$$

where $\varepsilon > 0$ is a small numerical stability constant that prevents division by zero when observed speeds approach zero. MAPE is scale-invariant and facilitates comparisons across datasets with different speed ranges or normalization conventions. However,

it has a known instability in traffic data: when observed speeds are near zero, even small absolute errors produce very large percentage values, inflating the metric in a way that does not reflect practically significant prediction degradation. For this reason, MAPE is reported for completeness in all results tables, but MAE and RMSE are the exclusive basis for all performance comparisons, ablation rankings, and architectural conclusions in this paper.

Original-scale metrics. Normalization is used for optimization, but all reported MAE, RMSE, and MAPE values are calculated after inverse transformation into the original speed scale. Training-period statistics are reused without re-estimation for validation and test windows. This avoids mixing normalized pilot scores with operationally interpretable full-data results.

3.5 Problem Scope and Assumptions

Several assumptions govern the problem formulation used in this paper and are stated explicitly here to support reproducibility and fair comparison.

Closed-world sensor network. The model is trained and evaluated on a fixed set of $N = 207$ METR-LA sensors or $N = 325$ PeMS-BAY sensors. No sensor addition, removal, or failure is modeled. All sensors are assumed to provide valid observations at every time step after standard preprocessing.

Fixed sampling frequency. All observations are at a fixed five-minute interval. Values encoded as unavailable are masked from the objective and metrics; irregular sampling and variable-rate prediction remain outside scope.

Single-feature input. Both benchmarks provide speed as the modeled feature. The model architecture supports $F > 1$ feature dimensions through the batch tensor format in Eq. (4), but all verified experiments use $F = 1$.

Chronological train-validation-test split. Because the forecasting challenge is causal, the data is divided chronologically without being shuffled across split borders. During training or validation, the model is never exposed to future observations; instead, it only looks at historical data. Section 6 provides the split sizes utilised in each experimental configuration.

Protocol-specific inference. The reported component effects apply to the two datasets, model implementations, random seeds, and optimization schedule tested here. They are not interpreted as universal architectural laws.

4 Dataset Description and Dependency Structure

This section describes the two traffic-speed benchmarks, preprocessing, and exploratory sensor dependencies. These details provide the data context for the architecture in Section 5 and the evidence in Section 7.

4.1 METR-LA Dataset

METR-LA contains 34,272 five-minute speed observations from $N = 207$ loop detectors on Los Angeles County freeways. The approximately four-month series covers 2,856 hours, and speeds are measured in miles per hour. Table 2 summarizes both benchmarks and their window counts.

Table 2. Dataset and chronological raw-split/window summary for the full-data experiments.

Dataset	Sensors	Time steps	Sampling	Train windows	Validation windows	Test windows
METR-LA	207	34,272	5 min	23,967	3,404	6,832
PeMS-BAY	325	52,105	5 min	36,450	5,188	10,398

METR-LA was introduced alongside DCRNN[9] and has since served as one of the two standard benchmarks for graph-based traffic forecasting, alongside PeMS-BAY. Its 207-sensor configuration strikes a balance between network complexity and computational tractability, making it suitable for controlled ablation experiments that require training multiple model variants under identical conditions. The highway setting also means that speed dynamics are primarily governed by freeway flow theory, with congestion propagating upstream from bottlenecks in patterns that graph-based models are well suited to capture.

4.2 PeMS-BAY Dataset

PeMS-BAY contains 52,105 five-minute speed observations from 325 sensors in the San Francisco Bay Area freeway network. It provides an independent network with more sensors and a longer observation period than METR-LA. The same 12-input/12-output, stride-one, chronological 70/10/20 protocol is used for both datasets. It forms the independent second benchmark in the confirmatory evaluation.

4.3 Data Preprocessing

For each dataset, the raw timeline is partitioned chronologically into 70% training, 10% validation, and 20% test segments before window construction. Independent 12-input/12-output windows are then constructed with stride one inside each segment.

Consequently, no input or target observation belongs to more than one split. No random split or reduced-window subsampling is used. The exact window counts are reported in Table 2.

Normalization statistics are fitted only on valid observations from the training period. Values encoded as zero are treated as unavailable. Such inputs are replaced with the training mean, which becomes zero after normalization, and unavailable target positions are excluded from the loss and all metrics. The fitted statistics are applied unchanged to validation and test windows, and reported metrics are inverse-transformed to miles per hour.

The DCRNN-compatible METR-LA distance adjacency is taken from the public dataset release with MD5 `55996dfc e3e39cc981e6e42e3f427da8`; PeMS-BAY uses the adjacency supplied with the experiment archive. Reference graphs are used by the appropriate baselines and graph-agreement diagnostics. ASTAF-R itself learns its adaptive adjacency from node embeddings.

4.4 Exploratory dependency and temporal structure

Figure 1 presents an exploratory Pearson correlation visualization for the first 50 METR-LA sensors. It shows that traffic-speed dependencies are heterogeneous and motivates testing an adaptive graph. However, the figure covers only part of the network and a small set of sampled days; it is not used to establish whole-network graph generalization.

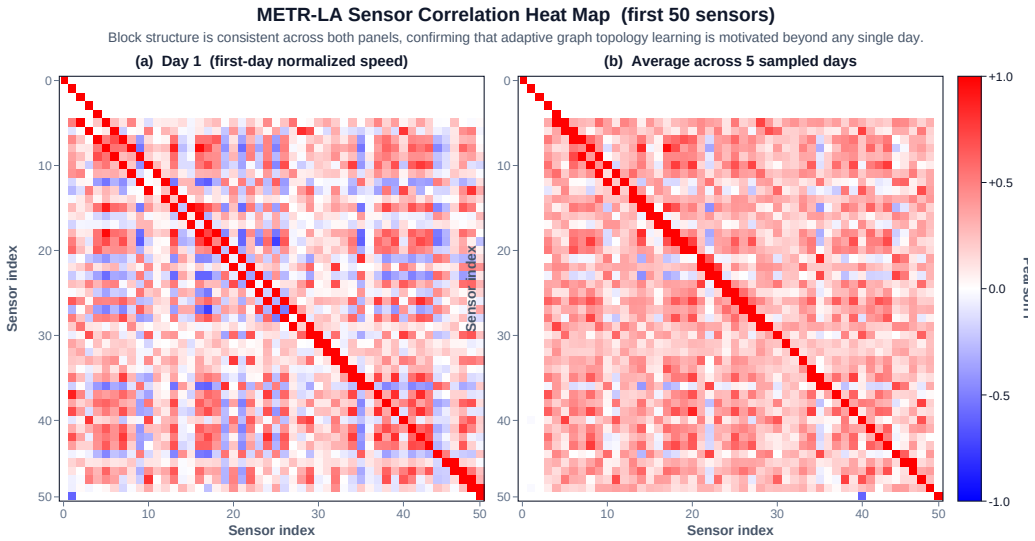


Figure 1. Exploratory Pearson correlation heat maps for the first 50 METR-LA sensors. Panel (a) uses the first day, and panel (b) averages five sampled days. The visualization motivates a learned topology but is insufficient by itself to establish whole-network stability; quantitative full-network diagnostics are reported in Section 7.5.

The five-minute sampling interval gives a one-hour 12-step input window. Short sensor fluctuations, approximately 15-minute congestion changes, and approximately 30-minute build-up or dissipation motivate candidate temporal kernel sizes $\{1,3,6\}$. These receptive fields are not assumed to be optimal: one-scale, kernel-3, and three-scale variants are compared in the sensitivity analysis. Similarly, periodic and slowly varying traffic behavior motivates testing trend, recurrent seasonal, and residual representations, but the contribution of decomposition and its fusion weights is determined experimentally rather than inferred from the exploratory plots.

5 ASTAF-R Framework

5.1 Overview

Figure 2 summarizes the deterministic forecasting path. For a batch of speed windows $\mathbf{X} \in \mathbb{R}^{B\times 12\times N\times 1}$, the network produces $\widehat{\mathbf{Y}} \in \mathbb{R}^{B\times 12\times N}$. No quantile or probabilistic output head is used in the evaluated model.

5.2 Ordered multi-scale temporal representation

For each temporal scale $q \in \{1,3,6\}$, a one-dimensional convolution acts independently on every sensor history. Even-width padding is cropped to retain exactly 12 causal input positions. After GELU activation and temporal pooling, the scale-specific tensors $\mathbf{Z}^{(q)} \in \mathbb{R}^{B\times N\times d}$ are combined by a sample- and sensor-specific softmax gate:

$$\alpha_{bnq} = \frac{\exp(a_{bnq})}{\sum_{j\in\{1,3,6\}} \exp(a_{bnj})}, \qquad \mathbf{Z}_{\mathrm{ms},bn} = \sum_{q\in\{1,3,6\}} \alpha_{bnq}\mathbf{Z}^{(q)}_{bn}. \tag{8}$$

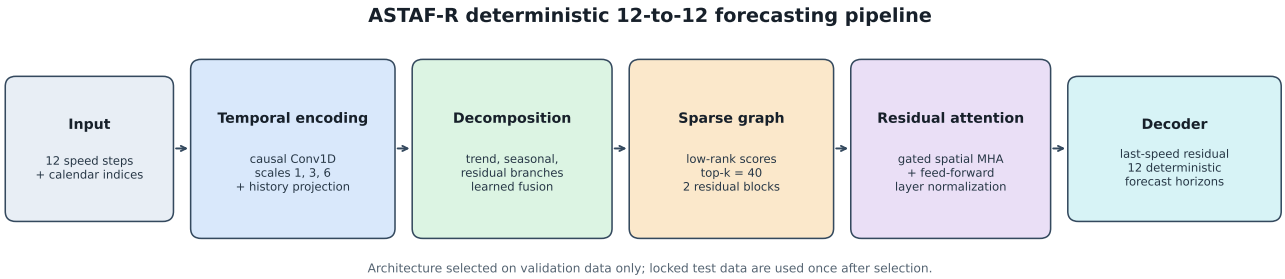


Figure 2. ASTAF-R deterministic forecasting pipeline. The candidate configuration was selected from validation results before locked testing.

Temporal pooling alone cannot distinguish histories with equal pooled summaries. The complete normalized history is therefore projected in its original order:

$$\mathbf{Z}_{\text{hist},bn} = \text{GELU}(\mathbf{W}_h \widetilde{\mathbf{x}}_{bn} + \mathbf{b}_h), \tag{9}$$

where $\widetilde{\mathbf{x}}_{bn} \in \mathbb{R}^{12}$ contains only observed input positions. The temporal state is $\mathbf{Z}_{\text{ms}} + \mathbf{Z}_{\text{hist}}$.

5.3 Sensor and calendar context

Every sensor has a learned embedding $\mathbf{e}_n \in \mathbb{R}^d$. The forecast origin determines a time-of-day index in $\{0,\dots,287\}$ and a day-of-week index in $\{0,\dots,6\}$. Their embeddings provide information available at forecast time:

$$\mathbf{Z}_{\text{ctx}} = \text{LN}(\mathbf{Z}_{\text{ms}} + \mathbf{Z}_{\text{hist}} + \mathbf{E}_{\text{node}} + \mathbf{E}_{\text{tod}} + \mathbf{E}_{\text{dow}}). \tag{10}$$

No target speed or calendar value derived from the target window enters the encoder.

5.4 Residual decomposition and learned fusion

The input history is mapped to trend, recurrent-seasonal, and residual representations $\mathbf{T}$, $\mathbf{S}$, and $\mathbf{R}$. The trend branch uses a width-7 reflected convolution; the recurrent branch averages the hidden states of a single GRU; and the residual summarizes what remains after the trend and recurrent summaries. Three trainable logits form non-negative fusion weights whose total contribution is fixed at 0.70:

$$\mathbf{w} = 0.70\,\text{softmax}(\ell), \qquad \mathbf{Z}_0 = \text{LN}(\mathbf{Z}_{\text{ctx}} + w_T\mathbf{T} + w_S\mathbf{S} + w_R\mathbf{R}). \tag{11}$$

The logits are initialized from $(0.35, 0.25, 0.10)$ and optimized on the training split. This construction supplies complementary feature summaries; it does not assert a uniquely identifiable physical decomposition. Fixed, equal, trend-heavy, and decomposition-disabled controls are evaluated on the validation split.

5.5 Directed sparse graph construction

Source and target node embeddings $\mathbf{E}_s, \mathbf{E}_t \in \mathbb{R}^{N\times d}$ define directed compatibility scores

$$s_{ij} = \frac{\text{ReLU}\left(\mathbf{e}_{s,i}^{\top}\mathbf{e}_{t,j}/\sqrt{d}\right)}{\tau}, \qquad \tau = 0.5. \tag{12}$$

Let $\mathscr{K}_{40}(i)$ denote the indices of the 40 largest scores in row i. Row-wise normalization gives

$$A_{ij} = \frac{\exp(s_{ij})\mathbb{I}[j \in \mathscr{K}_{40}(i)]}{\sum_{m\in\mathscr{K}_{40}(i)}\exp(s_{im})}. \tag{13}$$

The formulation permits an asymmetric adjacency and contains no manually selected identity-blending coefficient. The checkpoint diagnostic in Section 7.5 shows that the learned embeddings collapsed to zero in these runs; Eq. (13) consequently reduced to uniform sparse averaging on a deterministic support. The equation describes the implemented mechanism, while the diagnostic limits its empirical interpretation.

5.6 Residual graph propagation

Two graph blocks transform and aggregate node states while retaining a direct node-local path:

$$\mathbf{P}^{(\ell)} = \mathbf{A}\, g_\ell\left(\mathbf{Z}^{(\ell)}\right), \tag{14}$$

$$\mathbf{Z}^{(\ell+1)} = \mathrm{LN}\left(\mathbf{Z}^{(\ell)} + o_\ell\left(\mathrm{Dropout}(\mathrm{GELU}(\mathbf{P}^{(\ell)}))\right)\right), \qquad \ell \in \{0,1\}. \tag{15}$$

The residual connection prevents spatial aggregation from erasing the temporally encoded state. Validation controls vary graph depth, neighborhood size, and graph source rather than presuming that one graph formulation is universally preferable.

5.7 Residual spatial attention

Four-head self-attention operates over the N sensor tokens after graph propagation. Both attention and its feed-forward network are residual corrections:

$$\mathbf{Z}_a = \mathrm{LN}\left(\mathbf{Z}^{(2)} + \mathrm{MHA}(\mathbf{Z}^{(2)})\right), \tag{16}$$

$$\mathbf{Z}_f = \mathrm{LN}(\mathbf{Z}_a + \mathrm{FFN}(\mathbf{Z}_a)). \tag{17}$$

This residual design is distinct from mandatory replacement by an attention output. The two variants are directly compared during validation.

5.8 Deterministic multi-horizon decoder

A two-layer decoder produces all horizons jointly as corrections to the last observed normalized speed:

$$\widehat{\mathbf{Y}} = \mathbf{X}_{:,12,:,:} + \mathrm{reshape}\left(\mathrm{MLP}(\mathbf{Z}_f)\right), \qquad \widehat{\mathbf{Y}} \in \mathbb{R}^{B\times 12\times N\times 1}. \tag{18}$$

Direct decoding avoids autoregressive rollout. The evaluated model is a deterministic point forecaster and has no quantile or probabilistic output head.

5.9 Forward procedure

Algorithm 1 gives the complete data path. All steps operate on the input window or metadata available at the forecast origin.

Algorithm 1 ASTAF-R deterministic forward pass

Require: Normalized history $\mathbf{X}$; sensor and forecast-origin indices

1: Encode kernel widths 1, 3, and 6 and project the ordered history
2: Add sensor, time-of-day, and day-of-week embeddings
3: Form trend, recurrent-seasonal, and residual summaries
4: Fuse the summaries using learned non-negative weights
5: Construct directed row-normalized top-40 adjacency $\mathbf{A}$
6: Apply two residual graph propagation blocks
7: Apply residual four-head spatial attention and residual FFN
8: Decode 12 corrections and add the last observed speed
9: **return** $\widehat{\mathbf{Y}}$

5.10 Dimensional flow and parameterization

Table 3 records the tensor passed between stages. The temporal encoder works within each sensor history; graph propagation and attention act on sensor tokens only after temporal compression.

5.11 Training objective and computational complexity

For validity mask $M_{bhn} = \mathbb{I}[Y_{bhn} \neq 0]$, training minimizes

$$\mathscr{L}_{\mathrm{MAE}} = \frac{\sum_{b,h,n} M_{bhn}\left|Y_{bhn} - \widehat{Y}_{bhn}\right|}{\sum_{b,h,n} M_{bhn}}. \tag{19}$$

Adam uses learning rate 10^{-3}, weight decay 10^{-5}, gradient-norm clipping at 5, mixed precision, and validation checkpointing. The best validation checkpoint is restored before the test split is evaluated.

Table 3. Dimensional flow for batch size B, N sensors, hidden dimension $d = 64$, input length $T = 12$, and output horizon $H = 12$.

Stage	Output shape	Main trainable quantities
Input window	$B \times T \times N \times 1$	None
Scale encoders	$B \times N \times d$ per scale	Temporal convolutions and scale gate
Ordered history	$B \times N \times d$	$T \times d$ projection
Context	$B \times N \times d$	Node and calendar embeddings
Decomposition	Three $B \times N \times d$ tensors	Trend convolution, GRU, fusion logits
Sparse adjacency	$N \times N$	Source and target node embeddings
Graph and attention	$B \times N \times d$	Two graph blocks, four-head attention, FFN
Decoder	$B \times H \times N \times 1$	Direct multi-horizon MLP

With three temporal scales and two graph blocks, temporal encoding is approximately $\mathcal{O}(BNTd + BNd\sum_q q)$. Constructing the dense score matrix costs $\mathcal{O}(N^2 d)$. Although only 40 entries per row remain after selection, the reported implementation retains a dense $N \times N$ tensor; graph propagation and spatial attention therefore have a dominant $\mathcal{O}(BN^2 d)$ term. A sparse-kernel implementation could reduce message aggregation toward $\mathcal{O}(BNkd)$, but that optimization is not included in the reported latency or memory measurements.

6 Experimental Setup

6.1 Temporal boundaries and window construction

Each raw series was divided before window construction. METR-LA used raw index intervals $[0, 23990)$, $[23990, 27417)$, and $[27417, 34272)$ for training, validation, and testing. Their timestamp boundaries were 1 March 2012 00:00 to 23 May 2012 07:05, 23 May 2012 07:10 to 4 June 2012 04:40, and 4 June 2012 04:45 to 27 June 2012 23:55. PeMS-BAY used $[0, 36473)$, $[36473, 41683)$, and $[41683, 52105)$. Windows were then generated independently within each segment, yielding 23,967/3,404/6,832 METR-LA windows and 36,450/5,188/10,398 PeMS-BAY windows. Thus, the raw-observation intersection between every pair of splits is empty.

Zero speeds are treated as unavailable. Inputs are mean-imputed after the training-only global mean and standard deviation are computed; targets retain a mask so unavailable values do not contribute to loss or metrics. The forecast horizons are 15, 30, and 60 minutes at steps 3, 6, and 12, in addition to aggregate 12-step MAE, RMSE, and MAPE.

Table 4 records the first and last input and target timestamps formed inside every split. The one-step separation between the last target of a segment and the first input of the next segment confirms that no raw observation is reused across boundaries.

Algorithm 2 states the boundary procedure explicitly. A candidate start index is accepted only when its full 24-step input–target span is contained in one raw segment. This excludes all boundary-crossing starts that would otherwise share observations between two adjacent splits.

Algorithm 2 Leakage-free chronological splitting and window construction

Require: Raw series $\mathbf{V}_{0:L-1}$; input length $T = 12$; horizon $H = 12$
Set raw boundaries $b_0 = 0$, $b_1 = \lfloor 0.70L \rfloor$, $b_2 = \lfloor 0.80L \rfloor$, $b_3 = L$
for $r \in \{0, 1, 2\}$ **do**
    Initialize split $\mathscr{D}_r \leftarrow \varnothing$
    for $s = b_r$ **to** $b_{r+1} - T - H$ **do**
        $\mathbf{X} \leftarrow \mathbf{V}_{s:s+T-1}$
        $\mathbf{Y} \leftarrow \mathbf{V}_{s+T:s+T+H-1}$
        Append $(\mathbf{X}, \mathbf{Y}, s)$ to $\mathscr{D}_r$
    end for
end for
Fit normalization statistics on valid observations in $\mathbf{V}_{b_0:b_1-1}$ only
Apply them unchanged to $\mathscr{D}_0, \mathscr{D}_1, \mathscr{D}_2$
return training, validation, and test windows with raw indices

6.2 Selection and locked evaluation

Nine principal candidates were trained with seeds 11, 22, and 33 on both validation splits. Twenty broader graph, scale, decomposition, context, and cumulative controls used seed 11. Selection minimized the mean across-dataset relative validation MAE. The selected two-block, residual-attention, top-$k = 40$ configuration was frozen before any locked-test result was read. Every learned comparison model was then trained with seeds 11, 22, and 33; persistence is deterministic and appears once per dataset.

6.3 Comparison models and reproducibility scope

The controlled comparison contains compact common-code versions of STID, DCRNN, STGCN, a Transformer encoder, Graph WaveNet, AGCRN, and the initial ASTAF backbone. They share preprocessing, batching, loss, checkpoint selection, and evaluation code. They are not official checkpoints or exhaustive method-specific tuning studies, so differences are interpreted only within this protocol. Runs used Python 3.13.9, PyTorch 2.11.0 with CUDA 12.8, NumPy 2.5.1, and an NVIDIA GeForce RTX 5060 Laptop GPU with 7.96 GiB memory.

Table 5 makes the common-code scope auditable. Every learned model uses Adam, batch size 64, learning rate 10^{-3}, weight decay 10^{-5}, a 60-epoch cap, patience 10, and seeds 11, 22, and 33. Model-specific inductive biases are retained, whereas preprocessing, target masking, checkpoint selection, and evaluation remain identical.

Published numbers provide useful context but do not constitute a controlled comparison because data treatment, software, optimization, and reporting details differ. Table 6 therefore lists selected official-repository or original-paper horizon MAEs separately from the common-code experiment. No statistical ranking is inferred across these rows.

Table 4. Exact temporal endpoints of the independently constructed windows. PeMS-BAY timestamps include the recorded Pacific-time UTC offset; METR-LA timestamps are reported as stored in the dataset index.

Dataset and split	First input	First target	Last input	Last target
METR-LA train	2012-03-01 00:00	2012-03-01 01:00	2012-05-23 06:05	2012-05-23 07:05
METR-LA validation	2012-05-23 07:10	2012-05-23 08:10	2012-06-04 03:40	2012-06-04 04:40
METR-LA test	2012-06-04 04:45	2012-06-04 05:45	2012-06-27 22:55	2012-06-27 23:55
PeMS-BAY train	2017-01-01 00:55−08:00	2017-01-01 01:55−08:00	2017-05-07 16:15−07:00	2017-05-07 17:15−07:00
PeMS-BAY validation	2017-05-07 17:20−07:00	2017-05-07 18:20−07:00	2017-05-25 18:30−07:00	2017-05-25 19:30−07:00
PeMS-BAY test	2017-05-25 19:35−07:00	2017-05-25 20:35−07:00	2017-06-30 22:55−07:00	2017-06-30 23:55−07:00

Table 5. Principal settings of the compact common-code comparison models. The "style" suffix distinguishes these controls from official repositories and independently tuned published systems.

Model	Principal configuration	Spatial input
Persistence	Last valid value repeated over 12 horizons	None
STID-style	64-d history, node, and calendar embeddings; MLP decoder	None
DCRNN-style	64-d diffusion-GRU encoder; direct 12-step output	Reference adjacency
STGCN-style	Two 64-channel temporal/graph convolution stages	Reference adjacency
Transformer-style	64-d embedding; two encoder layers; four heads	None
Graph WaveNet-style	64 channels; dilations 1, 2, 4; graph convolution	Learned + reference
AGCRN-style	64-d adaptive graph recurrent encoder	Learned adjacency
ASTAF backbone	Multi-scale/decomposition/graph with mandatory attention	Learned adjacency
ASTAF-R	Selected architecture in Section 5	Learned top-40 graph

6.4 Statistical and computational analysis

We report sample mean and sample standard deviation across seeds. Seed-paired MAE differences are supplemented with exact sign-flip tests; with only three seeds, the minimum attainable two-sided exact value is 0.25 and is interpreted accordingly. Prediction windows are also aggregated into non-overlapping calendar-day blocks. A deterministic 10,000-repetition bootstrap resamples days within each seed and averages across the three seeds. Computational reporting includes parameter count, profiler FLOPs, synchronized inference latency, and peak allocated GPU memory.

Table 6. Non-harmonized literature context: horizon-wise MAE reported by the DCRNN official repository and the original Graph WaveNet and AGCRN papers[9,14,15], alongside ASTAF-R under the present protocol. These values are not treated as a head-to-head experiment.

		METR-LA			PeMS-BAY		
Source	Method	15 min	30 min	60 min	15 min	30 min	60 min
Official repository	DCRNN	2.67	3.08	3.56	1.31	1.66	1.98
Original paper	Graph WaveNet	2.69	3.07	3.53	1.30	1.63	1.95
Original paper	AGCRN	2.86	3.25	3.68	1.36	1.69	1.98
This study	ASTAF-R	2.717	3.031	3.400	1.204	1.404	1.875

7 Results and Discussion

7.1 Validation-only component selection

Table 7 shows the closest principal candidates. The top-$k = 20$ two-block model has slightly lower METR-LA validation MAE than the selected top-$k = 40$ model, whereas top-$k = 40$ is lower on PeMS-BAY. The across-dataset relative criterion selects top-$k = 40$ by a narrow margin (1.000691 versus 1.000794). The one-block model is not selected. Decomposition disabled, fixed-weight, and learned alternatives are close, so the evidence does not establish a unique decomposition fusion rule.

Figure 3 broadens the validation view beyond the six closest candidates in Table 7. Dark bars denote three-seed principal candidates and light bars denote seed-11 diagnostic controls. Several alternatives cluster closely, especially on PeMS-BAY; consequently, the plot supports a coordinated configuration choice rather than a claim that every selected component is independently dominant.

The ordered seed-11 diagnostic in Table 8 adds one component at a time using the same implementation and validation split. It shows the largest step at calendar context, followed by graph propagation and residual attention. Because cumulative ablations are order-dependent and this diagnostic uses one seed, it complements rather than replaces the three-seed principal candidate analysis.

7.2 Locked full-split comparison

Table 9 reports the locked test results. ASTAF-R has the lowest mean MAE, RMSE, and MAPE among the compact common-code models on both datasets. Relative to the strongest MAE comparison, STID-style, its mean MAE is 11.3% lower on METR-LA and 9.7% lower on PeMS-BAY. These numbers support superiority within the declared local pipeline only; they are not comparisons with official implementations or all published systems.

MAE alone can obscure whether a ranking depends on the error definition. Figure 5 normalizes MAE, RMSE, and MAPE by the best model value for each dataset and metric. ASTAF-R is the unit reference on all six dataset–metric combinations; the relative gaps are larger for the under-performing controls and remain visible for the closest comparison. Figure 6 summarizes the same locked results as ranks across the three metrics. These views do not add independent observations, but they make the consistency of the multi-metric comparison explicit.

7.3 Optimization trajectories

Figure 7 displays the saved training and validation histories for the three ASTAF-R seeds. The best checkpoint is selected at the minimum validation MAE, not at the final epoch. Similar trajectories across seeds are consistent with the small locked-test standard deviations, while the train–validation gap motivates retaining validation checkpointing and early stopping.

7.4 Seed and calendar-day variability

ASTAF-R is lower than every comparison for each matched seed. Exact seed sign-flip tests remain descriptive because $n = 3$ gives low resolution. The calendar-day block analysis in Table 10 uses 24 METR-LA and 37 PeMS-BAY day blocks per seed. Every 95% bootstrap interval for ASTAF-R minus a comparison remains below zero, showing that the aggregate differences are not driven by a small set of overlapping windows. Resampling whole days reduces the dependence induced by stride-one windows, but it does not remove serial dependence between adjacent days; a moving multi-day block analysis would be a useful additional robustness check.

7.5 Graph diagnostic

The cross-seed adjacency correlation and top-20 Jaccard values are exactly one to numerical precision (Table 11). Inspection of the six restored best checkpoints explains this result: both graph-embedding matrices have zero norm, all pre-softmax graph

Table 7. Validation-only evidence for principal architecture choices. Standard deviations are across seeds 11, 22, and 33.

Dataset	Candidate	Seeds	Validation MAE (mph)
METR-LA	Fixed decomposition weights	3	2.775±0.007
	Decomposition disabled	3	2.773±0.011
	One graph block, top-$k = 20$	3	2.780±0.008
	No residual output	3	2.785±0.013
	Two graph blocks, top-$k = 20$	3	2.771±0.003
	Two graph blocks, top-$k = 40$	3	2.774±0.006
PeMS-BAY	Fixed decomposition weights	3	1.571±0.003
	Decomposition disabled	3	1.577±0.009
	One graph block, top-$k = 20$	3	1.576±0.007
	No residual output	3	1.596±0.009
	Two graph blocks, top-$k = 20$	3	1.574±0.009
	Two graph blocks, top-$k = 40$	3	1.571±0.006

Table 8. Cumulative validation MAE for an ordered seed-11 diagnostic. Each row retains all components in the preceding row.

Cumulative configuration	METR-LA	PeMS-BAY
Temporal encoder	3.456	2.197
+ ordered history	3.442	2.185
+ calendar context	3.064	1.787
+ decomposition	3.062	1.784
+ graph propagation	2.964	1.709
+ residual attention	2.776	1.598
+ residual decoder	2.770	1.565

logits are zero, and deterministic top-k tie handling selects the same 40 indices per row. Each selected edge therefore has weight $1/40 = 0.025$. This is a degenerate uniform sparse graph, not evidence that independently initialized adaptive topologies converged to a stable road-network structure. The correct conclusion is that sparse graph averaging is compatible with the selected model; learned-topology recovery is not validated by these runs.

The uniformity is also visible in Table 12. Entropy equals $\log(40)$, top-10 mass is $10/40$, and the largest edge weight is $1/40$. Correlation and top-20 overlap with the reference graph are near zero, so the deterministic support is neither a quantitative reconstruction of the provided road graph nor evidence of meaningful learned neighborhoods.

7.6 Forecast horizons and engineering trade-offs

Error increases with horizon on both datasets (Figure 9), while ASTAF-R retains the lowest mean MAE in the local comparison at all 12 horizons. The accuracy comes with higher parameter count and greater inference memory/latency than STID-, DCRNN-, and STGCN-style controls (Table 13). ASTAF-R is therefore not the preferred choice when minimum memory or latency dominates accuracy; deployment selection should use the application-specific trade-off.

Figure 10 visualizes the same deployment trade-off. ASTAF-R uses fewer profiler operations than several graph-recurrent controls but more parameters and substantially more memory and latency than STID- and STGCN-style controls. Its error advantage therefore does not imply universal deployment preference.

7.7 Scope of the evidence

The experiments cover two California freeway-speed datasets with regular five-minute sampling and no exogenous weather, event, or incident variables. They do not establish transfer to other cities, irregular sampling, flow or occupancy targets, distribution shift, or heterogeneous multi-frequency covariates. The deterministic evaluation also does not establish calibrated uncertainty forecasting. The broader sensitivity inventory includes several single-scale and paired-scale controls, but not a scale-6-only model or every possible receptive-field set; it therefore supports robustness around the chosen configuration rather than a unique optimum. Calendar-context removal and the non-principal graph controls are seed-11 diagnostics, so they are not assigned the same evidential weight as the three-seed principal candidates.

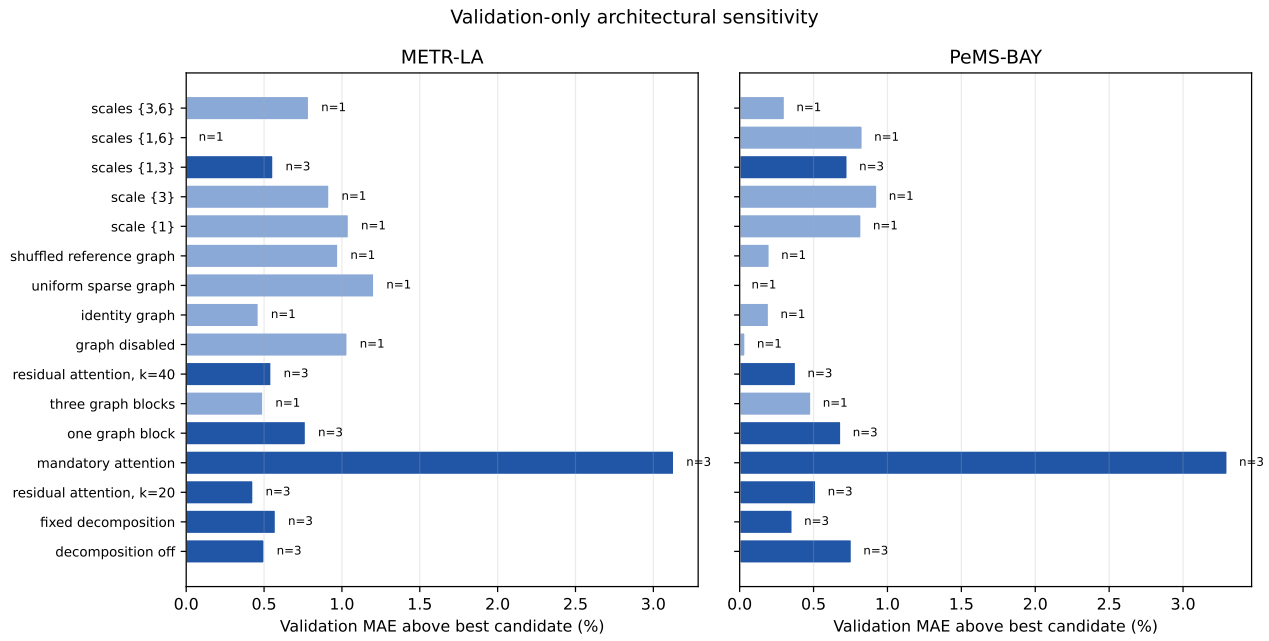


Figure 3. Validation-only architectural sensitivity, expressed as the percentage by which each candidate's validation MAE exceeds the best displayed candidate within that dataset. The annotation gives the number of seeds; no locked-test metric is used in this figure.

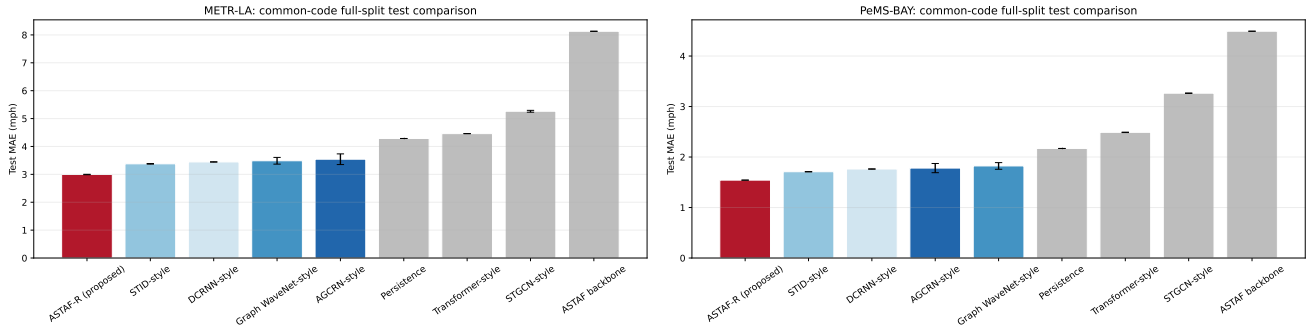


Figure 4. Locked-test MAE for the compact common-code comparison. Error bars show sample standard deviation across seeds.

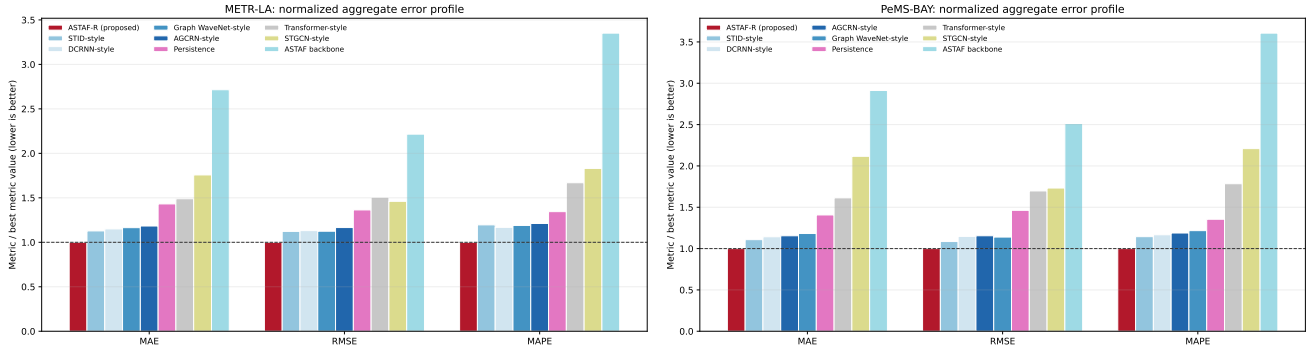


Figure 5. Locked-test MAE, RMSE, and MAPE normalized by the lowest mean value for each metric and dataset. A value of 1 denotes the best mean result; error bars propagate the across-seed sample standard deviation through the same normalization.

Table 9. Locked-test performance under a common leakage-free protocol. Values are mean±sample standard deviation across three seeds, except deterministic persistence. Lower is better.

Dataset	Model	MAE (mph)	RMSE (mph)	MAPE (%)
METR-LA	Persistence	4.286	8.214	11.167
	STID-style	3.377±0.011	6.761±0.018	9.931±0.012
	DCRNN-style	3.445±0.007	6.813±0.036	9.700±0.103
	STGCN-style	5.262±0.033	8.797±0.060	15.203±0.083
	Transformer-style	4.460±0.001	9.071±0.006	13.866±0.025
	Graph WaveNet-style	3.487±0.121	6.773±0.280	9.878±0.396
	AGCRN-style	3.542±0.192	7.022±0.441	10.063±0.605
	ASTAF backbone	8.130±0.003	13.347±0.062	27.836±0.164
	ASTAF-R	**2.995±0.008**	**6.024±0.010**	**8.306±0.024**
PeMS-BAY	Persistence	2.170	5.123	4.671
	STID-style	1.709±0.003	3.801±0.002	3.945±0.016
	DCRNN-style	1.762±0.003	4.018±0.024	4.023±0.036
	STGCN-style	3.264±0.004	6.068±0.013	7.617±0.043
	Transformer-style	2.489±0.004	5.945±0.002	6.152±0.046
	Graph WaveNet-style	1.823±0.066	3.990±0.204	4.196±0.175
	AGCRN-style	1.781±0.091	4.045±0.304	4.097±0.192
	ASTAF backbone	4.491±0.001	8.803±0.026	12.433±0.024
	ASTAF-R	**1.542±0.001**	**3.503±0.009**	**3.448±0.025**

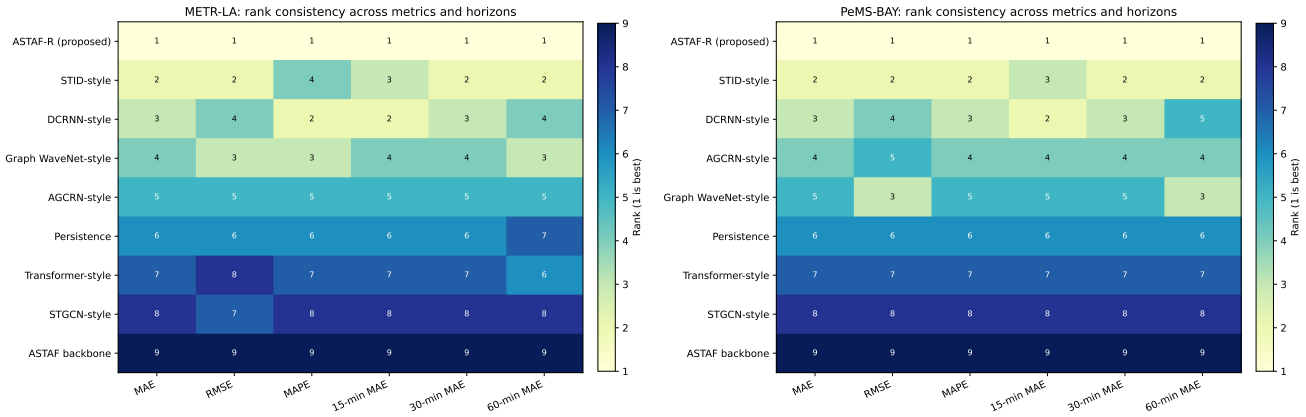


Figure 6. Metric-wise ranks of the compact common-code models on the locked test splits. Lower ranks are better; ties, if present, receive their average rank.

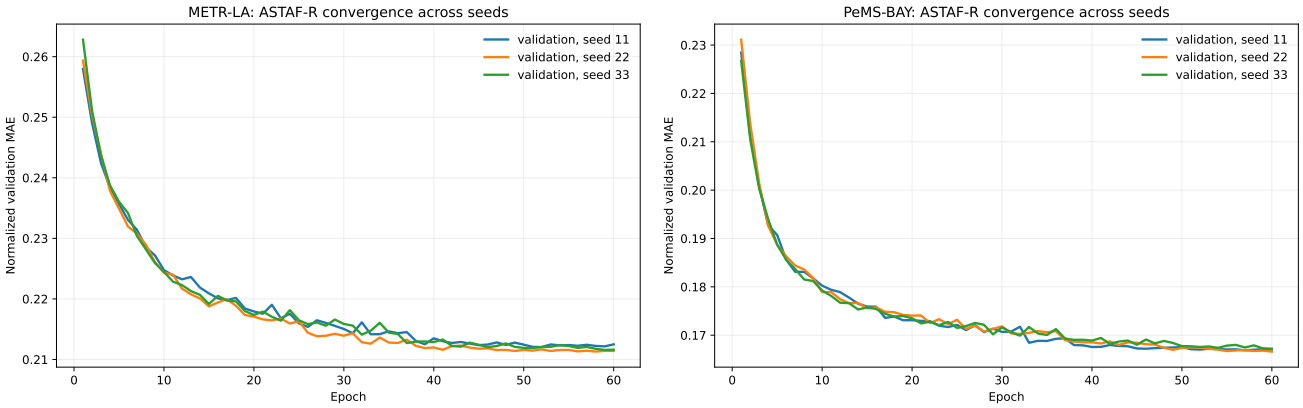


Figure 7. ASTAF-R training and validation MAE histories for seeds 11, 22, and 33. Curves are taken from the stored run histories, and each locked test run uses the corresponding minimum-validation checkpoint.

Table 10. Calendar-day-block MAE differences. Negative values favor ASTAF-R; intervals are 10,000-repetition clustered day-bootstrap intervals.

Dataset	Comparison	Mean difference	95% block CI	Seed SD
METR-LA	vs. AGCRN-style	-0.520	[-0.573, -0.470]	0.174
	vs. ASTAF backbone	-4.751	[-5.057, -4.425]	0.008
	vs. DCRNN-style	-0.429	[-0.461, -0.397]	0.011
	vs. Graph WaveNet-style	-0.467	[-0.507, -0.428]	0.109
	vs. STGCN-style	-2.104	[-2.221, -1.981]	0.028
	vs. STID-style	-0.363	[-0.395, -0.332]	0.009
	vs. Transformer-style	-1.362	[-1.490, -1.227]	0.006
PeMS-BAY	vs. AGCRN-style	-0.236	[-0.262, -0.209]	0.090
	vs. ASTAF backbone	-2.904	[-3.103, -2.707]	0.001
	vs. DCRNN-style	-0.218	[-0.239, -0.196]	0.003
	vs. Graph WaveNet-style	-0.278	[-0.304, -0.251]	0.065
	vs. STGCN-style	-1.701	[-1.783, -1.617]	0.004
	vs. STID-style	-0.164	[-0.180, -0.148]	0.004
	vs. Transformer-style	-0.931	[-1.036, -0.825]	0.005

Table 11. Cross-seed graph diagnostics reported without rounding away the exact identity. The identity is explained by zero graph embeddings and uniform top-40 weights.

Dataset	Seed pair	Pearson correlation	Top-20 Jaccard
METR-LA	11–22	1.000000000000	1.000000000000
METR-LA	11–33	1.000000000000	1.000000000000
METR-LA	22–33	1.000000000000	1.000000000000
PeMS-BAY	11–22	1.000000000000	1.000000000000
PeMS-BAY	11–33	1.000000000000	1.000000000000
PeMS-BAY	22–33	1.000000000000	1.000000000000

Table 12. Within-graph diagnostics for the selected top-40 support. Correlation and Jaccard compare the learned-weight matrix with the supplied reference adjacency; values are averaged over the restored seed checkpoints.

Dataset	Row entropy	Top-10 mass	Max. weight	Corr. with reference	Top-20 Jaccard
METR-LA	3.688879	0.250000	0.025000	0.004383	0.026558
PeMS-BAY	3.688879	0.250000	0.025000	−0.004980	0.015622

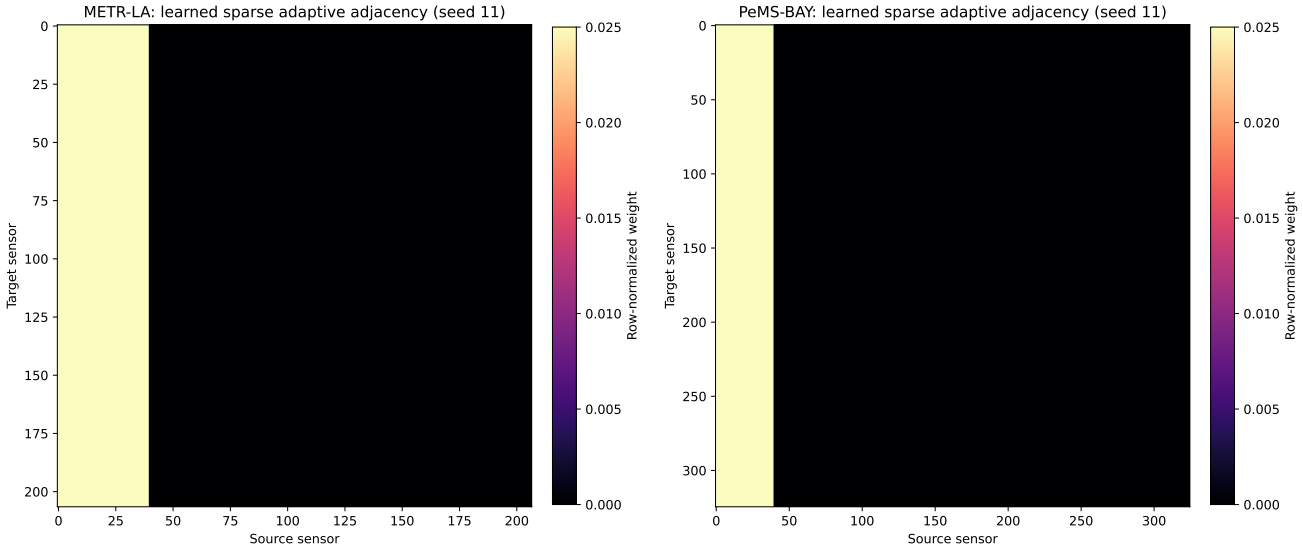


Figure 8. Sparse graph matrices from seed 11 for METR-LA (left) and PeMS-BAY (right). Uniform nonzero weights and deterministic support must not be interpreted as recovered road-network topology.

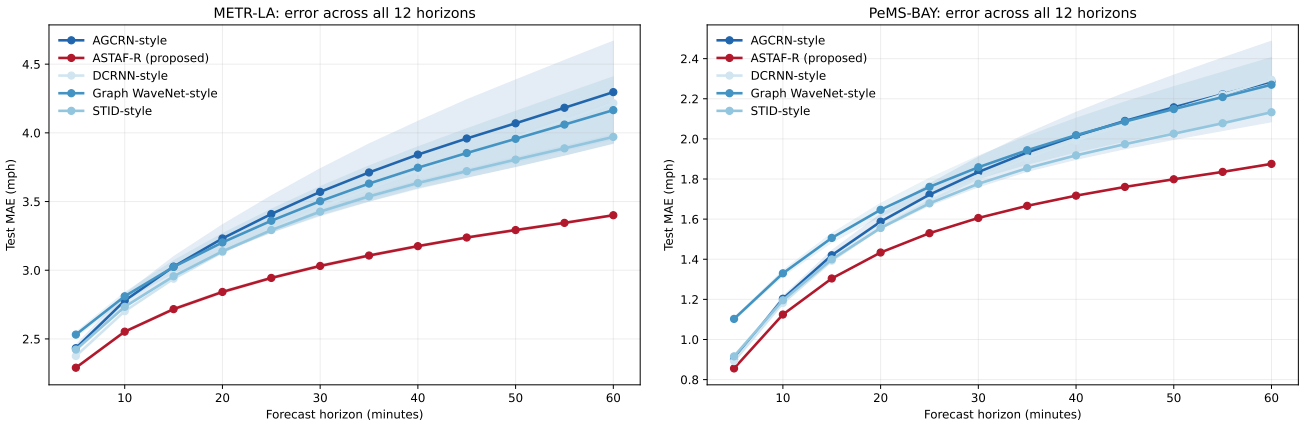


Figure 9. MAE across the 12 five-minute forecast horizons. Shading denotes sample standard deviation across seeds.

Table 13. Environment-specific computational profile. Latency and peak memory are not hardware-independent model properties.

Dataset	Model	Parameters	FLOPs	Latency (ms/sample)	Peak GPU (MiB)
METR-LA	ASTAF-R	133,842	52,053,945	0.184	377.9
	STID-style	38,028	2,358,209	0.017	28.8
	DCRNN-style	38,412	454,333,536	0.233	48.9
	Graph WaveNet-style	114,765	1,344,774,051	0.253	303.9
	AGCRN-style	60,493	896,704,128	0.200	55.4
	Persistence	0	–	0.001	18.5
	Transformer-style	100,876	341,480,448	0.430	412.9
	STGCN-style	13,516	193,950,720	0.051	115.8
	ASTAF backbone	91,984	752,433,408	0.193	377.0
PeMS-BAY	ASTAF-R	156,498	106,271,169	0.312	578.2
	STID-style	45,580	3,702,465	0.028	33.8
	DCRNN-style	38,412	952,629,600	0.385	66.1
	AGCRN-style	75,597	2,100,009,600	0.217	109.1
	Graph WaveNet-style	129,869	2,921,427,275	0.423	467.1
	Persistence	0	–	0.002	19.2
	Transformer-style	100,876	536,140,800	0.686	637.0
	STGCN-style	13,516	422,323,200	0.095	171.1
	ASTAF backbone	107,088	1,839,136,000	0.358	576.5

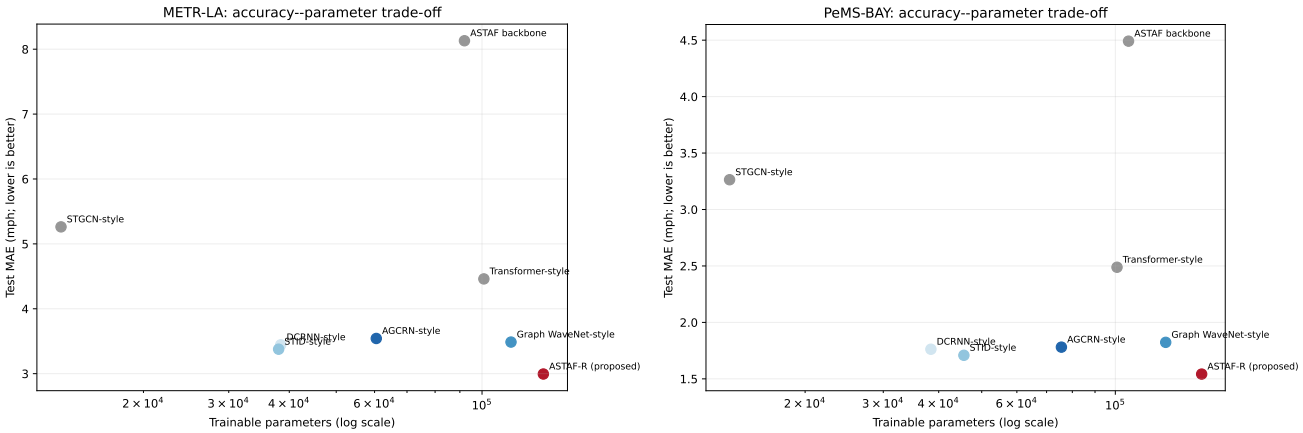


Figure 10. Accuracy–efficiency profiles measured on the stated hardware. Points compare locked-test MAE with synchronized inference latency, peak allocated GPU memory, parameter count, and profiler operations; these engineering measurements are environment-specific.

8 Conclusion and Future Work

ASTAF-R was evaluated using validation-only architecture selection and a locked, leakage-free test protocol on METR-LA and PeMS-BAY. It achieved the lowest mean error among the compact common-code controls on both datasets, with small between-seed dispersion and calendar-day bootstrap intervals favoring ASTAF-R. The conclusion is deliberately protocol-specific: official baseline implementations were not executed, and two California freeway datasets cannot support a universal generality claim.

The diagnostic evidence also changes the interpretation of the graph module. The graph embeddings converged to zero, producing an identical uniform top-40 support across seeds. The present results therefore support sparse graph propagation as part of the forecasting computation, but they do not validate adaptive topology recovery. Similarly, closely matched decomposition variants do not identify one fusion rule as uniquely effective.

Future work should compare official baseline repositories under harmonized splits, repair or regularize the graph learner to avoid zero-embedding collapse, repeat selection after that change, and evaluate geographically distinct and irregularly sampled networks with exogenous variables. Probabilistic extensions should be trained with proper scoring rules and evaluated using coverage, interval width, calibration error, and distributional scores.

9 Reproducibility Record

The public archive contains preprocessing, model, training, evaluation, and baseline code; configuration values; random seeds; environment dependencies; run-level JSON and NumPy artifacts; generated analysis tables; and commands for reproducing the study. Exact raw-split boundaries and the absence of cross-split raw-observation overlap are recorded in every locked-test run.

9.1 Horizon-wise numerical results

Figure 9 gives the complete trend; Table 14 provides the corresponding values for ASTAF-R and the closest aggregate-MAE comparison. The difference column is ASTAF-R minus STID-style, so negative values favor ASTAF-R. The difference remains negative at every horizon on both datasets and generally widens as the forecast range increases. This table is a numerical expansion of the locked analysis, not a separate tuning experiment.

Tables 15 and 16 extend the numerical record to every compact comparison model. They report mean MAE to keep the matrices readable; the corresponding across-seed dispersion appears as shading in Figure 9 and remains available in the machine-readable horizon table.

9.2 Complete validation candidate inventory

Tables 17 and 18 report every stored candidate summary used in the component analysis. The identifier is preserved verbatim so that each row can be matched directly to its configuration and run artifacts. Principal decisions were repeated over three seeds; broader diagnostic controls used seed 11 and are marked with a seed count of one. Single-seed rows are not assigned an artificial standard deviation and are not treated as equally strong evidence.

9.3 Seed-aware and calendar-block inference

Let $e_{s,w}^{(m)}$ be per-window MAE for model m, seed s, and test window w. Because stride-one windows overlap strongly, window-level errors are not treated as independent replicates. Windows whose forecast origins fall on the same calendar day g are first aggregated:

$$d_{s,g}^{(m)} = \frac{1}{|\mathcal{W}_g|} \sum_{w \in \mathcal{W}_g} \left(e_{s,w}^{(\text{ASTAF-R})} - e_{s,w}^{(m)} \right). \tag{20}$$

Each bootstrap repetition resamples calendar days with replacement within each seed and then averages the three seed-specific means:

$$\Delta_m^{*(r)} = \frac{1}{3} \sum_{s \in \{11,22,33\}} \frac{1}{G} \sum_{j=1}^{G} d_{s,g_j^{*(r)}}^{(m)}. \tag{21}$$

The 2.5th and 97.5th empirical percentiles of 10,000 deterministic repetitions form the intervals in Table 10. The separate seed-paired exact sign-flip test enumerates all 2^3 sign assignments. Its smallest attainable two-sided value is 0.25, which is why effect sizes, between-seed dispersion, and block-bootstrap intervals carry more information than the seed-level p-value alone.

Table 14. Locked-test horizon-wise MAE in mph for ASTAF-R and the closest aggregate-MAE compact control. Values are mean±sample standard deviation across seeds 11, 22, and 33.

Dataset	Horizon	ASTAF-R MAE	STID-style MAE	Difference
METR-LA	1 (5 min)	2.290±0.005	2.423±0.006	-0.132
	2 (10 min)	2.553±0.007	2.734±0.005	-0.181
	3 (15 min)	2.717±0.009	2.956±0.006	-0.239
	4 (20 min)	2.842±0.009	3.137±0.008	-0.295
	5 (25 min)	2.944±0.008	3.292±0.009	-0.348
	6 (30 min)	3.031±0.010	3.426±0.011	-0.395
	7 (35 min)	3.107±0.009	3.538±0.013	-0.431
	8 (40 min)	3.175±0.008	3.635±0.014	-0.459
	9 (45 min)	3.238±0.007	3.721±0.015	-0.483
	10 (50 min)	3.292±0.008	3.805±0.016	-0.513
	11 (55 min)	3.345±0.007	3.887±0.016	-0.542
	12 (60 min)	3.400±0.005	3.970±0.016	-0.570
PeMS-BAY	1 (5 min)	0.856±0.000	0.916±0.006	-0.060
	2 (10 min)	1.124±0.001	1.196±0.005	-0.071
	3 (15 min)	1.304±0.001	1.399±0.003	-0.095
	4 (20 min)	1.434±0.001	1.556±0.003	-0.122
	5 (25 min)	1.530±0.001	1.679±0.003	-0.149
	6 (30 min)	1.606±0.001	1.776±0.003	-0.170
	7 (35 min)	1.666±0.001	1.854±0.003	-0.188
	8 (40 min)	1.717±0.000	1.918±0.003	-0.201
	9 (45 min)	1.761±0.001	1.974±0.004	-0.213
	10 (50 min)	1.799±0.001	2.026±0.003	-0.227
	11 (55 min)	1.835±0.002	2.078±0.003	-0.242
	12 (60 min)	1.875±0.003	2.133±0.003	-0.257

9.4 Artifact-to-claim traceability

Table 19 links major claims to machine-readable outputs. Run files retain dataset identifiers, raw boundaries, window counts, seed, environment, epoch history, best checkpoint epoch, test metrics, parameter count, profiler operations, latency, and peak memory. NumPy archives store aligned start indices, per-window errors, adjacency matrices, and decomposition weights used by the downstream analyses.

Table 15. METR-LA locked-test mean MAE at all 12 horizons for every compact common-code model. GWN denotes Graph WaveNet-style, Trans. denotes Transformer-style, and Persist. denotes persistence.

Horizon	ASTAF-R	STID	DCRNN	GWN	AGCRN	STGCN	Trans.	ASTAF	Persist.
1 (5 min)	2.290	2.423	2.376	2.531	2.433	4.380	3.357	8.097	2.832
2 (10 min)	2.553	2.734	2.702	2.810	2.778	4.576	3.598	8.102	3.238
3 (15 min)	2.717	2.956	2.938	3.024	3.026	4.751	3.819	8.108	3.534
4 (20 min)	2.842	3.137	3.133	3.202	3.231	4.915	4.028	8.114	3.790
5 (25 min)	2.944	3.292	3.301	3.360	3.409	5.069	4.228	8.120	4.025
6 (30 min)	3.031	3.426	3.455	3.502	3.570	5.221	4.419	8.127	4.255
7 (35 min)	3.107	3.538	3.592	3.631	3.712	5.366	4.600	8.133	4.472
8 (40 min)	3.175	3.635	3.722	3.746	3.842	5.508	4.773	8.140	4.677
9 (45 min)	3.238	3.721	3.845	3.853	3.959	5.645	4.940	8.146	4.870
10 (50 min)	3.292	3.805	3.967	3.956	4.070	5.777	5.102	8.152	5.062
11 (55 min)	3.345	3.887	4.090	4.060	4.183	5.905	5.256	8.158	5.247
12 (60 min)	3.400	3.970	4.216	4.165	4.296	6.031	5.405	8.166	5.431

Table 16. PeMS-BAY locked-test mean MAE at all 12 horizons for every compact common-code model. Abbreviations follow Table 15.

Horizon	ASTAF-R	STID	DCRNN	GWN	AGCRN	STGCN	Trans.	ASTAF	Persist.
1 (5 min)	0.856	0.916	0.892	1.103	0.901	2.813	1.635	4.474	0.972
2 (10 min)	1.124	1.196	1.182	1.330	1.203	2.903	1.830	4.477	1.327
3 (15 min)	1.304	1.399	1.393	1.506	1.420	2.991	2.007	4.480	1.591
4 (20 min)	1.434	1.556	1.557	1.646	1.587	3.078	2.170	4.483	1.809
5 (25 min)	1.530	1.679	1.691	1.761	1.722	3.160	2.322	4.486	1.998
6 (30 min)	1.606	1.776	1.805	1.859	1.835	3.240	2.466	4.489	2.171
7 (35 min)	1.666	1.854	1.904	1.943	1.932	3.319	2.602	4.492	2.333
8 (40 min)	1.717	1.918	1.991	2.019	2.015	3.393	2.731	4.496	2.486
9 (45 min)	1.761	1.974	2.071	2.086	2.090	3.465	2.854	4.499	2.631
10 (50 min)	1.799	2.026	2.147	2.148	2.157	3.535	2.971	4.502	2.771
11 (55 min)	1.835	2.078	2.220	2.209	2.222	3.602	3.083	4.506	2.906
12 (60 min)	1.875	2.133	2.295	2.270	2.286	3.668	3.191	4.511	3.039

Table 17. Complete METR-LA validation-candidate inventory. Rows are ordered by validation MAE; candidate identifiers match the public configuration and artifact names.

Candidate identifier	Seeds	Validation MAE (mph)
plus_scales16_64	1	2.759
cumulative_residual64	1	2.770
plus_resattn64	3	2.771±0.003
plus_correlation_graph64	1	2.772
plus_weights_trend64	1	2.772
plus_identity_graph64	1	2.772
plus_graph3_64	1	2.773
plus_decomp_off64	3	2.773±0.011
plus_topk40_64	3	2.774±0.006
plus_scales13_64	3	2.775±0.017
plus_decomp_fixed64	3	2.775±0.007
cumulative_attention64	1	2.776
plus_graph1_64	3	2.780±0.008
plus_scales36_64	1	2.781
plus_weights_equal64	1	2.781
plus_scales3_64	1	2.785
plus_no_residual64	3	2.785±0.013
plus_shuffled_graph64	1	2.786
plus_no_graph64	1	2.788
plus_scales1_64	1	2.788
plus_uniform_graph64	1	2.793
plus_no_context64	1	2.820
plus_mandatory64	3	2.846±0.017
plus_fast64	3	2.952±0.004
cumulative_graph64	1	2.964
cumulative_decomposition64	1	3.062
cumulative_context64	1	3.064
cumulative_history64	1	3.442
cumulative_temporal64	1	3.456

Table 18. Complete PeMS-BAY validation-candidate inventory. Rows are ordered by validation MAE; candidate identifiers match the public configuration and artifact names.

Candidate identifier	Seeds	Validation MAE (mph)
plus_weights_trend64	1	1.562
cumulative_residual64	1	1.565
plus_uniform_graph64	1	1.565
plus_no_graph64	1	1.566
plus_identity_graph64	1	1.569
plus_shuffled_graph64	1	1.569
plus_scales36_64	1	1.570
plus_decomp_fixed64	3	1.571±0.003
plus_topk40_64	3	1.571±0.006
plus_graph3_64	1	1.573
plus_resattn64	3	1.574±0.009
plus_graph1_64	3	1.576±0.007
plus_scales13_64	3	1.577±0.005
plus_decomp_off64	3	1.577±0.009
plus_correlation_graph64	1	1.577
plus_scales1_64	1	1.578
plus_scales16_64	1	1.578
plus_weights_equal64	1	1.580
plus_scales3_64	1	1.580
plus_no_residual64	3	1.596±0.009
cumulative_attention64	1	1.598
plus_no_context64	1	1.612
plus_mandatory64	3	1.617±0.006
plus_fast64	3	1.656±0.001
cumulative_graph64	1	1.709
cumulative_decomposition64	1	1.784
cumulative_context64	1	1.787
cumulative_history64	1	2.185
cumulative_temporal64	1	2.197

Table 19. Traceability from manuscript evidence to public archive artifacts.

Evidence	Primary machine-readable artifact	Verification purpose
Raw split isolation	locked run JSON files	Boundaries, counts, zero overlap
Validation selection	`validation_candidates.csv`	Candidate means, seeds, criterion
Locked aggregate metrics	`main_results.csv`	MAE, RMSE, MAPE and dispersion
Horizon curves	`horizon_results.csv`	Steps 1–12 for every model
Seed comparisons	`seed_level_tests.csv`	Paired effects and sign-flip tests
Calendar-block intervals	`day_block_bootstrap.csv`	Day counts and bootstrap intervals
Graph diagnosis	`graph_results.csv` and NPZ files	Norms, supports, weights, correlations
Efficiency profile	run JSON profiler fields	Parameters, operations, latency, memory

Author contributions statement

S.S. Chauhan conceived the study, contributed to the research design, and led the initial manuscript preparation. P. Nema contributed to data preprocessing, model implementation, and experimental execution. G. Chhabra assisted in methodological refinement, validation, and interpretation of the experimental results. A. Pandey supervised the study, contributed to result analysis, visualization, critical manuscript editing, and coordinated the study. S. Syal contributed to literature analysis, manuscript editing, and interpretation of the findings. All authors reviewed the manuscript, provided intellectual input, approved the final version, and agreed to be accountable for the content of the work. Correspondence should be addressed to A. Pandey.

Competing interests

The authors declare no competing interests.

Funding Declaration

The authors received no specific funding for this work.

Data Availability

METR-LA and PeMS-BAY are publicly available from the Zenodo dataset releases at `https://doi.org/10.5281/zenodo.5724362` and `https://doi.org/10.5281/zenodo.5146275`.

Code Availability

The complete source code, preprocessing, model and compact baseline implementations, configuration values, dependencies, seeds, training and evaluation commands, and analysis scripts are publicly available at `https://github.com/shishirbit/ASTAF-R-reproducibility`. The matching versioned reproducibility package has the reserved Zenodo DOI `https://doi.org/10.5281/zenodo.23611484`.

References

1. Box, G. E. P. & Jenkins, G. M. *Time Series Analysis: Forecasting and Control* (Holden-Day, San Francisco, CA, USA, 1970).
2. Hochreiter, S. & Schmidhuber, J. Long short-term memory. *Neural Comput.* **9**, 1735–1780, DOI: 10.1162/neco.1997.9.8.1735 (1997).
3. Cho, K. *et al.* Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, 1724–1734, DOI: 10.3115/v1/D14-1179 (2014).
4. Zhang, J., Zheng, Y. & Qi, D. Deep spatio-temporal residual networks for citywide crowd flows prediction. In *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 31, DOI: 10.1609/aaai.v31i1.10735 (2017).
5. Yao, H. *et al.* Deep multi-view spatial-temporal network for taxi demand prediction. In *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, DOI: 10.1609/aaai.v32i1.11898 (2018).
6. Pan, Z. *et al.* Urban traffic prediction from spatio-temporal data using deep meta learning. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1720–1730, DOI: 10.1145/3292500.3330884 (2019).

7. Kipf, T. N. & Welling, M. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations* (2017).

8. Veličković, P. *et al.* Graph attention networks. In *International Conference on Learning Representations* (2018).

9. Li, Y., Yu, R., Shahabi, C. & Liu, Y. Diffusion convolutional recurrent neural network: Data-driven traffic forecasting. In *International Conference on Learning Representations* (2018).

10. Yu, B., Yin, H. & Zhu, Z. Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting. In *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence*, 3634–3640, DOI: 10.24963/ijcai.2018/505 (2018).

11. Cui, Z., Henrickson, K., Ke, R. & Wang, Y. High-order graph convolutional recurrent neural network: A deep learning framework for network-scale traffic learning and forecasting. *IEEE Transactions on Intell. Transp. Syst.* **21**, 4883–4894, DOI: 10.1109/TITS.2019.2950416 (2020).

12. Li, Y., Zheng, Z., Zhang, S. & Huang, H. Traffic scene modeling with spatial-temporal graphical model. *IEEE Transactions on Intell. Transp. Syst.* **20**, 3078–3089, DOI: 10.1109/TITS.2018.2873649 (2019).

13. Song, C., Lin, Y., Guo, S. & Wan, H. Spatial-temporal synchronous graph convolutional networks: A new framework for spatial-temporal network data forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, 914–921, DOI: 10.1609/aaai.v34i01.5438 (2020).

14. Wu, Z., Pan, S., Long, G., Jiang, J. & Zhang, C. Graph WaveNet for deep spatial-temporal graph modeling. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence*, 1907–1913, DOI: 10.24963/ijcai.2019/264 (2019).

15. Bai, L., Yao, L., Li, C., Wang, X. & Wang, C. Adaptive graph convolutional recurrent network for traffic forecasting. *Adv. Neural Inf. Process. Syst.* **33**, 17804–17815 (2020).

16. Shao, Z. *et al.* Decoupled dynamic spatial-temporal graph neural network for traffic forecasting. *Proc. VLDB Endow.* **15**, 2733–2746, DOI: 10.14778/3551793.3551806 (2022).

17. Sun, M., Huang, W., Zhou, Z. & Ye, W. Adaptive neighborhood selection for spatio-temporal graph neural networks in traffic forecasting. *IEEE Transactions on Intell. Transp. Syst.* **25**, 2574–2586, DOI: 10.1109/TITS.2023.3321034 (2024).

18. Liu, Y., Liu, Y., Huang, W. & Zeng, L. Spatio-temporal graph attention convolution network for traffic flow forecasting. *Expert. Syst. with Appl.* **242**, 122746, DOI: 10.1016/j.eswa.2023.122746 (2024).

19. Cui, Z., Henrickson, K., Ke, R. & Wang, Y. Traffic graph convolutional recurrent neural network: A deep learning framework for network-scale traffic learning and forecasting. *IEEE Transactions on Intell. Transp. Syst.* **21**, 4883–4894, DOI: 10.1109/TITS.2019.2950416 (2020).

20. Ye, J., Sun, L., Du, B., Fu, Y. & Xiong, H. Coupled layer-wise graph convolution for transportation demand forecasting. *IEEE Transactions on Intell. Transp. Syst.* **25**, 1630–1641, DOI: 10.1109/TITS.2023.3323450 (2024).

21. Zhao, L., Chen, C., Wang, F. & Deng, L. Frequency-enhanced heterogeneous graph neural network for traffic flow prediction. *IEEE Transactions on Intell. Transp. Syst.* **25**, 187–199, DOI: 10.1109/TITS.2023.3299514 (2024).

22. Zheng, G., Liu, C., Wei, H. & Li, Z. Knowledge-informed machine learning for cancer diagnosis and prognosis: A survey. DOI: 10.1109/TNNLS.2023.3264001 (2023).

23. Brimos, P., Argyriou, A. & Chatzigiannakis, I. Open government data and artificial intelligence: A systematic literature review on benefits, challenges and future directions. *Futur. Internet* **15**, 114, DOI: 10.3390/fi15030114 (2023).

24. Vaswani, A. *et al.* Attention is all you need. In *Advances in Neural Information Processing Systems*, vol. 30 (2017).

25. Guo, S., Lin, Y., Feng, N., Song, C. & Wan, H. Attention based spatial-temporal graph convolutional networks for traffic flow forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, 922–929, DOI: 10.1609/aaai.v33i01.3301922 (2019).

26. Zheng, C., Fan, X., Wang, C. & Qi, J. GMAN: A graph multi-attention network for traffic prediction. In *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, 1234–1241, DOI: 10.1609/aaai.v34i01.5477 (2020).

27. Zheng, C., Fan, X., Wang, C. & Qi, J. GMAN: A graph multi-attention network for traffic prediction. In *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, 1234–1241, DOI: 10.1609/aaai.v34i01.5477 (2020).

28. Zhang, W., Chen, C., Yu, L. & Gao, C. Spatio-temporal adaptive graph convolutional networks for traffic flow forecasting. *IET Intell. Transp. Syst.* **17**, 1–13, DOI: 10.1049/itr2.12246 (2023).

29. Han, L. *et al.* Dynamic and multi-faceted spatio-temporal deep learning for traffic speed forecasting. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 547–555, DOI: 10.1145/3447548.3467275 (2021).

30. Wu, H., Xu, J., Wang, J. & Long, M. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. In *Advances in Neural Information Processing Systems*, vol. 34, 22419–22430 (2021).

31. Zhou, T. *et al.* FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting. In *Proceedings of the 39th International Conference on Machine Learning*, vol. 162 of *Proceedings of Machine Learning Research*, 27268–27286 (PMLR, 2022).

32. Xiao, C. & Liu, Y. A multifrequency data fusion deep learning model for carbon price prediction. *J. Forecast.* **44**, 436–458, DOI: 10.1002/for.3198 (2025).

33. Yin, X. *et al.* Deep learning on traffic prediction: Methods, analysis, and future directions. *IEEE Transactions on Intell. Transp. Syst.* **23**, 4927–4943, DOI: 10.1109/TITS.2021.3054840 (2022).

34. Jiang, W. & Luo, J. Graph neural network for traffic forecasting: A survey. *Expert. Syst. with Appl.* **207**, 117921, DOI: 10.1016/j.eswa.2022.117921 (2022).

35. Shao, Z., Zhang, Z., Wang, F., Wei, W. & Xu, Y. Spatial-temporal identity: A simple yet effective baseline for multivariate time series forecasting. In *Proceedings of the 31st ACM International Conference on Information and Knowledge Management*, 4454–4458, DOI: 10.1145/3511808.3557702 (2022).